

SnowFLOWS: Generative Spatio-Temporal Modelling of Snow Cover Dynamics Using Satellite Imagery

By:

Carla Youness (232771)
Nour el Saghir (230520)
Reina Taher (230134)
Ali Mansour (231039)
Hassan Dakhallallah (222814)

Supervisor(s):

Dr. Laurent Drapeau
Dr. Youssef El Bakouny

11/04/2026

École Supérieure d'Ingénieurs de Beyrouth
Université Saint-Joseph de Beyrouth

1. Recap of Previous Progress Phase

During the previous phase of the project, we focused on building the initial version of the snow prediction dataset and establishing a baseline modeling workflow. Satellite-derived snow coverage data from Sentinel-2 and MODIS were collected and processed across multiple altitude ranges in Lebanon. These datasets were cleaned, structured into weekly observations, and aligned temporally to create a consistent time-series format suitable for machine learning applications.

2. Construction of the Multi-Source Snow Prediction Dataset

Following the preparation of the initial satellite snow coverage dataset in the previous phase, the dataset was extended by integrating additional meteorological variables obtained from the ERA5 climate reanalysis database. The objective of this step was to improve the model's ability to capture environmental conditions influencing snow accumulation.

The final dataset was organized in a weekly time-series format using the `week_start` variable as the temporal reference. For each week, snow coverage values derived from both Sentinel-2 and MODIS were included across three altitude ranges:

- Above 1200 m
- Between 1200 m and 1800 m
- Above 1800 m

To improve predictive performance, additional atmospheric variables were incorporated from ERA5:

- Air temperature (°C)
- Total precipitation (mm)

These meteorological variables were also structured according to the same altitude ranges to ensure spatial consistency with the satellite-derived snow observations.

As a result, each weekly observation contains synchronized information describing:

- Snow surface area from Sentinel-2
- Snow surface area from MODIS
- Temperature conditions
- Precipitation levels

across multiple elevation bands. This multi-source structure allows the predictive model to better learn the relationship between terrain elevation, seasonal atmospheric behavior, and snow cover evolution.

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	Sentinel-2			Modis			Temperature-Celsius			Precipitation-mm			
2	week_start	snow_km2 - above 1200	snow_km2 - between 1200 - 1800	snow_km2 - above 1800	snow_km2 - above 1200	snow_km2 - between 1200 - 1800	snow_km2 - above 1800	Temp- 1200	Temp-1200-1800	Temp-1800	precipitation-1200	precipitation-1200-1800	precipitation-1800
3	01/10/2016	0	0	0	0	0	0	21.55524133	18.92267723	16.90063696	1.509897396	0.905635894	1.445756874
4	08/10/2016	0	0	0	0.012	0	0	20.75631131	18.13326995	16.06664512	4.338199341	5.189557857	5.453085127
5	15/10/2016	0	0	0	0	0	0	19.11294656	16.29739278	14.29842438	8.095914794	5.748534774	9.294732547
6	22/10/2016	0	0	0	0.319	2	0.428	18.58097367	15.52727249	13.27790686	98.32805688	149.9597844	166.8631627
7	29/10/2016	0	0	0	13.756	9.002	5.097	15.7558068	12.21013624	10.03367962	309.8818154	237.9029424	176.0646099
8	05/11/2016	0	0	0	0.598	0.357	0.047	15.3782425	12.3621018	10.60984648	6.530988564	7.462474448	6.191659517
9	12/11/2016	0	0	0	0.938	0.204	0.216	15.41298684	12.74611863	11.12820265	27.45592797	19.23234648	13.55640923
10	19/11/2016	0	0	0	0.953	0.095	0	12.13423187	8.283599278	6.433315633	0.329386629	0.640266854	0.689458963
11	26/11/2016	0	0	0	150.354	96.933	80.99	10.11719601	6.281987217	4.303367653	731.9398443	573.5285239	456.8984195

Structure of the multi-source weekly dataset before normalization showing Sentinel-2, MODIS, ERA5 temperature, and precipitation variables across altitude bands

63	25/11/2017	184.1997126	0.309311375	183.8904013	51.129	1.599	130.422	11.22526454	7.508950236	5.397942683	81.38830839	57.10284543	50.82788848
64	02/12/2017	1123.075177	574.0606472	549.01453	28.205	160.297	298.505	10.58674306	7.403819692	5.755394097	336.0419733	373.670389	345.4033621
65	09/12/2017	0	0	0	1.202	115.219	537.248	10.56408441	7.59680388	5.997491249	5.22485632	2.688173769	1.127011973
66	16/12/2017	632.5158645	303.6603085	328.855556	11.271	17.788	159.999	13.32382402	10.69664078	8.95819024	5.401476463	6.100544131	7.877861435
67	23/12/2017	19.82435067	2.10793498	17.71641569	34.504	236.69	484.784	10.34741824	7.151963968	5.591900868	423.4100257	328.7187646	282.9536048
68	30/12/2017	20.03008682	6.454187984	13.57589884	262.539	83.373	92.801	8.645132503	5.073440728	3.170839171	426.3748951	395.652176	339.7477946
69	06/01/2018	392.6643373	77.31673489	315.3476024	6.132	87.363	495.892	8.988493179	5.952275159	4.613299991	42.66681024	47.27291759	37.1279599
70	13/01/2018	746.8383529	37.25722459	709.5811283	126.517	42.625	311.576	7.52582605	3.624402823	1.59641663	662.3386504	534.3621984	408.130608
71	20/01/2018	1485.797989	742.8948318	742.9031576	102.595	1.01042	838.938	7.287054123	3.374124295	1.409488926	759.4929366	649.1842406	541.3642643
72	27/01/2018	1343.004459	297.4366985	1045.56776	11.737	219.237	807.406	6.943543266	3.429397102	1.571013367	31.16652868	24.72274221	20.56998241
73	03/02/2018	0	0	0	1.589	74.176	556.867	12.9125184	10.66219391	8.912307055	7.341640572	8.490989192	9.88718142
74	10/02/2018	0	0	0	1.473	8.413	368.317	11.11582891	7.801441589	5.945998164	324.6521939	293.0417424	230.3294223
75	17/02/2018	10.39224634	0	10.39224634	10.631	196.779	851.326	9.445359615	5.954989218	3.816306687	632.3679555	578.1414616	548.8812583
76	24/02/2018	359.6583993	2.681754094	356.9766452	6.376	6.758	399.646	9.635093156	6.054121952	3.865765275	312.1245401	292.1401518	255.2774526
77	03/03/2018	257.993167	0	257.993167	0.473	0.859	218.915	13.91181551	11.09817947	9.124980741	14.72288786	12.83517027	10.6211273
78	10/03/2018	187.915095	0	187.915095	1.466	0.028	125.009	11.38166591	8.536936518	6.818538969	66.60133439	59.50858742	47.69843453
79	17/03/2018	139.7706573	0	139.7706573	0.198	0	84.921	15.75225147	13.16183539	11.37683054	3.172818917	2.885586117	2.45080182
80	24/03/2018	76.99364557	0	76.99364557	0.246	0.089	58.788	14.05008653	11.22591672	9.546103721	276.1108855	259.9461549	197.3651653
81	31/03/2018	135.7876016	50.54109007	85.2465115	0.192	0.004	52.534	14.72197101	12.2637071	10.53428679	7.213272193	5.793079746	3.7548943
82	07/04/2018	13.35354732	2.422485574	10.93106175	0.21	0.934	23.255	13.946174	11.27593941	9.080336089	194.8326162	222.2670287	277.8552417
83	14/04/2018	28.44816679	0	28.44816679	0	0	12.493	17.52800113	15.19685795	13.05808378	1.263497527	0.904959734	0.936134062
84	21/04/2018	11.59268649	3.063010188	8.529676302	0.296	0.319	7.404	15.34905173	12.3885261	10.30467266	257.3259132	206.2815456	221.2095849
85	28/04/2018	30.85748222	13.70816777	17.14931445	0.442	3.396	8.44	19.50394678	15.87503511	13.34447384	129.5611415	198.0744914	208.1154979
86	05/05/2018	266.2352201	222.5153918	43.71982832	2.311	0	5.351	17.13043581	14.06077206	11.88897852	414.644299	399.4388047	379.2804897

Example winter-period observations illustrating the relationship between precipitation increase and snow surface expansion across elevation levels

3. Temporal Structuring and Feature Standardization

To prepare the dataset for sequence-based machine learning modeling, the original week_start column was transformed into two separate temporal variables:

- year
- week_of_year

This transformation allows the model to better capture seasonal snow behavior patterns across multiple winters while preserving chronological ordering of observations. Representing time explicitly in this format improves the learning capacity of recurrent neural networks such as LSTM models, which rely on sequential temporal dependencies.

After extracting these variables, the original week_start column was removed and replaced by the newly generated time indicators positioned at the beginning of the dataset.

	A	B
1	year	week_of_year
2	2016	39
3	2016	40
4	2016	41
5	2016	42
6	2016	43
7	2016	44
8	2016	45
9	2016	46
10	2016	47
11	2016	48
12	2016	49
13	2016	50
14	2016	51
15	2016	52
16	2017	1
17	2017	2
18	2017	3
19	2017	4

Conversion of week_start into year and week_of_year temporal features

4. Feature Scaling Using Z-Score Standardization

Because the dataset integrates variables originating from different physical sources and measurement units (snow surface area, temperature, and precipitation), feature scaling was required before training the predictive model.

A Z-score normalization procedure was applied using the StandardScaler function from the scikit-learn library. This transformation standardizes each feature according to:

$$Z = \frac{X - \mu}{\sigma}$$

where:

- X is the original feature value
- μ is the feature mean
- σ is the feature standard deviation

This ensures that all variables share the same statistical scale, preventing features with larger numeric ranges (such as precipitation values) from dominating the learning process.

The normalization was applied to all satellite and meteorological variables across altitude bands, including:

- Sentinel-2 snow coverage
- MODIS snow coverage
- Temperature measurements
- Precipitation measurements

The standardized dataset was then saved as: DataSet_after_zscore.csv

and used as the main input dataset for the predictive modeling phase.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
	year	week_of_year	s2_snow_above_1200 smoothed_zscore	s2_snow_1200_1800 smoothed_zscore	s2_snow_above_1800 smoothed_zscore	modis_snow_above_1200 smoothed_zscore	modis_snow_1200_1800 smoothed_zscore	modis_snow_above_1800 smoothed_zscore	temp_1200_smoother smoothed_zscore	temp_1200_1800 smoothed_zscore	temp_1800_sm smoothed_zscore	precip_1200_sm smoothed_zscore	precip_1200_1800 smoothed_zscore	precip_1800_sm smoothed_zscore
1	2016	39	-0.485215972	-0.348455749	-0.56845063	-0.316612423	-0.379117441	-0.665794697	0.759025708	0.703774242	0.674355353	-0.691234716	-0.709094203	-0.740918343
2	2016	40	-0.485215972	-0.348455749	-0.56845063	-0.316648024	-0.379117441	-0.665794697	0.640395965	0.597696515	0.564744055	-0.677649322	-0.685404519	-0.715503501
3	2016	41	-0.485215972	-0.348455749	-0.56845063	-0.316612423	-0.379117441	-0.665794697	0.396379678	0.350997905	0.33247331	-0.659599606	-0.68231343	-0.691139423
4	2016	42	-0.485215972	-0.348455749	-0.56845063	-0.312773826	-0.371111685	-0.664329521	0.317389272	0.247511894	0.198221018	-0.226180801	0.115161157	0.308173927
5	2016	43	-0.485215972	-0.348455749	-0.56845063	-0.151083464	-0.343083533	-0.648346098	-0.102107818	-0.198233014	-0.2281668	0.789991209	0.601478513	0.366530344
6	2016	44	-0.485215972	-0.348455749	-0.56845063	-0.309416558	-0.377688413	-0.665633801	-0.158826793	-0.177812426	-0.152441362	-0.667116533	-0.672835505	-0.710819395
7	2016	45	-0.485215972	-0.348455749	-0.56845063	-0.305325263	-0.378300854	-0.665055262	-0.153011706	-0.126209617	-0.084313965	-0.566606205	-0.607749225	-0.664111481
8	2016	46	-0.485215972	-0.348455749	-0.56845063	-0.305144765	-0.378737167	-0.665794697	-0.639860176	-0.725867011	-0.701361564	-0.696905152	-0.710561669	-0.745714853
9	2016	47	-0.485215972	-0.348455749	-0.56845063	1.492630163	0.00893529	-0.388541017	-0.939361307	-0.994836483	-0.981299992	2.817294145	2.457456271	2.147603969
10	2016	48	-0.485215972	-0.348455749	-0.56845063	-0.246157926	0.024296603	2.181623525	-1.152035621	-1.195172554	-1.191286742	-0.29199064	-0.169706315	-0.092407922
11	2016	49	-0.485215972	-0.348455749	-0.56845063	1.401117534	0.973443014	1.780261253	-1.34818897	-1.372537154	-1.351055593	2.774667272	2.539971482	2.525396128
12	2016	50	-0.485215972	-0.348455749	-0.56845063	0.998004703	1.981607858	2.480348189	-1.693375571	-1.723747295	-1.771114832	0.844282729	1.255905503	1.585608336
13	2016	51	-0.485215972	-0.348455749	-0.56845063	0.923037749	2.368614105	2.328750437	-1.509184131	-1.539849146	-1.539581672	1.40597026	1.041423785	0.820803828
14	2016	52	-0.485215972	-0.348455749	-0.56845063	0.844208133	2.107398298	2.187990877	-1.627964023	-1.656145288	-1.663400513	0.5469248	0.364790181	0.243683471
15	2017	1	-0.219477084	0.000538042	-0.413985233	4.22103017	3.344367646	2.299378458	-1.631492466	-1.687584228	-1.635705103	0.858240289	0.758245462	0.492386312
16	2017	2	-0.485215972	-0.348455749	-0.56845063	-0.188350343	1.142212355	2.139759897	-1.384052814	-1.362627969	-1.314260794	-0.155583478	-0.011347416	0.021759259
17	2017	3	-0.485215972	-0.348455749	-0.56845063	0.283267603	0.679551714	1.85510845	-1.488528623	-1.484391107	-1.453510476	1.063554352	1.051961013	0.978996684
18	2017	4	-0.485215972	-0.348455749	-0.56845063	7.692132478	5.138209371	2.274884693	-2.3385263	-2.38507781	-2.348576725	0.388212252	0.545864728	0.651028191

Example of dataset variables after Z-score normalization

5. Definition of the Prediction Target Variable

The objective of the predictive model is to estimate weekly snow surface evolution in mountainous regions. For this purpose, the variable:

Sentinel-2 snow surface above 1200 m

was selected as the prediction target.

This variable was chosen because:

1. Elevations above 1200 m represent the primary snow accumulation zone in Lebanon
2. Sentinel-2 provides higher spatial resolution compared to MODIS
3. Snow persistence above this altitude strongly reflects seasonal snowfall dynamics
4. This altitude band is the most relevant for hydrological monitoring and winter-season analysis

To enable one-week-ahead forecasting, a shifted target column was created:
target_snow_next_week

This column contains the snow surface value of the following week relative to each observation. The shift operation allows the LSTM model to learn temporal relationships between current atmospheric conditions and future snow accumulation behavior.

The final row of the dataset was removed because it does not contain a valid next-week observation.

P
target_snow_next_week
-0.485215972
-0.485215972
-0.485215972
-0.485215972
-0.485215972
-0.485215972
-0.485215972
-0.485215972
-0.485215972

Creation of next-week prediction target column for snow surface above 1200 m

6. Train-Validation-Test Dataset Partitioning

To evaluate model performance reliably and prevent information leakage between training phases, the dataset was divided into three subsets:

Dataset	Percentage
Training set	70%
Validation set	15%
Test set	15%

A new column named: split

was introduced to explicitly label each observation according to its role in the modeling pipeline.

This structure allows:

- model training on historical sequences
- hyperparameter tuning using validation data
- final performance evaluation on unseen test observations

Such a partitioning strategy ensures that the predictive performance assessment remains unbiased and representative of real forecasting conditions.

C		
split	validation	test
train	validation	test
train	validation	test
train	validation	test
train	validation	test
train	validation	test
train	validation	test
train	validation	test

Dataset partitioning into training, validation, and testing subsets

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	year	week_of_year	split	s2_snow_ab	s2_snow_12	s2_snow_ab	modis_snow	modis_snow	modis_snow	temp_1200	temp_1200	temp_1800	precip_1200	precip_1200	precip_1800	target_snow_next_week
2	2016	39	train	-0.485216	-0.3484557	-0.5684506	-0.3166124	-0.3791174	-0.6657947	0.75902571	0.70377424	0.67435535	-0.6912347	-0.7090942	-0.7409183	-0.485215972
3	2016	40	train	-0.485216	-0.3484557	-0.5684506	-0.316468	-0.3791174	-0.6657947	0.64039597	0.59769652	0.56474406	-0.6776493	-0.6854045	-0.7155035	-0.485215972
4	2016	41	train	-0.485216	-0.3484557	-0.5684506	-0.3166124	-0.3791174	-0.6657947	0.39637968	0.35099791	0.33234733	-0.6595996	-0.6823134	-0.6911394	-0.485215972
5	2016	42	train	-0.485216	-0.3484557	-0.5684506	-0.3127738	-0.3711117	-0.6643295	0.31738927	0.24751189	0.19822102	-0.2261808	0.11516116	0.30817393	-0.485215972
6	2016	43	train	-0.485216	-0.3484557	-0.5684506	-0.1510835	-0.3430835	-0.6483461	-0.1021078	-0.198233	-0.2281668	0.78999121	0.60147851	0.36653034	-0.485215972
7	2016	44	train	-0.485216	-0.3484557	-0.5684506	-0.3094166	-0.3776884	-0.6656338	-0.1588268	-0.1778124	-0.1524414	-0.6671165	-0.6728355	-0.7108194	-0.485215972
8	2016	45	train	-0.485216	-0.3484557	-0.5684506	-0.3053253	-0.3783009	-0.6650553	-0.1530117	-0.1262096	-0.084314	-0.5666062	-0.6077492	-0.6641115	-0.485215972
9	2016	46	train	-0.485216	-0.3484557	-0.5684506	-0.3051448	-0.3787372	-0.6657947	-0.6398602	-0.725867	-0.7013616	-0.6969052	-0.7105617	-0.7457149	-0.485215972
10	2016	47	train	-0.485216	-0.3484557	-0.5684506	1.49263016	0.00889353	-0.388541	-0.9393613	-0.9948365	-0.9813	2.81729415	2.45745627	2.14760397	-0.485215972
11	2016	48	train	-0.485216	-0.3484557	-0.5684506	-0.2461579	0.0242966	2.18162353	-1.1520356	-1.1951726	-1.1912867	-0.2919906	-0.1697063	-0.0924079	-0.485215972
12	2016	49	train	-0.485216	-0.3484557	-0.5684506	1.40111753	0.97344301	1.78026125	-1.348189	-1.3725372	-1.3510556	2.77466727	2.53997148	2.52539613	-0.485215972
13	2016	50	train	-0.485216	-0.3484557	-0.5684506	0.9980047	1.98160786	2.48034819	-1.6933756	-1.7237473	-1.7711148	0.84428273	1.2559055	1.58560834	-0.485215972
14	2016	51	train	-0.485216	-0.3484557	-0.5684506	0.92303775	2.36861411	2.32875044	-1.5091841	-1.5398491	-1.5395817	1.40597026	1.04142379	0.82080383	-0.485215972
15	2016	52	train	-0.485216	-0.3484557	-0.5684506	0.84420813	2.1073983	2.18799088	-1.627964	-1.6561453	-1.6634005	0.5469248	0.36479018	0.24368347	-0.219477084
16	2017	1	train	-0.2194771	0.00053804	-0.4139852	4.22103017	3.34436765	2.29937846	-1.6314925	-1.6875842	-1.6357051	0.85824029	0.75824546	0.49238631	-0.485215972
17	2017	2	train	-0.485216	-0.3484557	-0.5684506	-0.1883503	1.14221236	2.1397599	-1.3840528	-1.362628	-1.3142608	-0.1555835	-0.0113474	0.02175926	-0.485215972
18	2017	3	train	-0.485216	-0.3484557	-0.5684506	0.2832676	0.67955171	1.85510845	-1.4885286	-1.4843911	-1.4535105	1.06355435	1.05196101	0.97899668	-0.485215972
19	2017	4	train	-0.485216	-0.3484557	-0.5684506	7.69213248	5.13820937	2.27488469	-2.385263	-2.3850778	-2.3485767	0.38821225	0.54586473	0.65102819	-0.483517456
20	2017	5	train	-0.4835175	-0.3484557	-0.565251	0.4029139	2.94495648	2.36341827	-1.3607617	-1.3832302	-1.3891129	-0.6661254	-0.668208	-0.6913844	-0.485215972
21	2017	6	train	-0.485216	-0.3484557	-0.5684506	0.10881	1.31202244	2.0692603	-1.7782442	-1.7846951	-1.822693	0.97810052	1.22180828	1.43255802	-0.485215972
22	2017	7	train	-0.485216	-0.3484557	-0.5684506	-0.291812	0.99151601	1.88080037	-1.4624764	-1.4072254	-1.3851619	-0.6943693	-0.7107767	-0.7453618	-0.485215972
23	2017	8	train	-0.485216	-0.3484557	-0.5684506	-0.299802	0.37165034	1.55661315	-0.844876	-0.844374	-0.8579396	0.70187921	0.75549845	0.58394541	-0.485215972
24	2017	9	train	-0.485216	-0.3484557	-0.5684506	-0.3153008	0.05038736	1.28779792	-0.8317718	-0.8277491	-0.8302553	-0.4762695	-0.4656429	-0.4112958	-0.485215972
25	2017	10	train	-0.485216	-0.3484557	-0.5684506	0.22926252	-0.2941243	0.25355167	-1.1487156	-1.1642614	-1.1660724	2.17923868	2.28435015	2.29066248	-0.485215972
26	2017	11	train	-0.485216	-0.3484557	-0.5684506	-0.288912	-0.3341411	0.76772904	-0.8947661	-0.8623785	-0.8655286	-0.278397	-0.0389015	0.06591489	-0.485215972
27	2017	12	train	-0.485216	-0.3484557	-0.5684506	-0.3027742	-0.3539754	0.30516145	-0.7015624	-0.6741306	-0.6978794	-0.3898463	-0.1927816	-0.0240314	0.145835408
28	2017	13	train	0.14583541	-0.0450239	0.3193802	-0.3006925	-0.3760713	-0.020556	-0.5687462	-0.5934775	-0.6102247	0.15859282	0.10245513	0.15565417	-0.477512882
29	2017	14	train	-0.4775129	-0.3484557	-0.5539395	-0.3114983	-0.3765236	-0.3936452	-0.4116237	-0.4256375	-0.4054697	0.41356895	0.94338532	1.03555125	-0.351947122

Final structured dataset after temporal feature extraction, Z-score normalization, target variable creation, and train–validation–test splitting ready for LSTM modeling

7. Preparation of Sequential Input Windows

Since Long Short-Term Memory (LSTM) networks operate on ordered temporal sequences rather than independent observations, the dataset was transformed into sliding input windows representing multiple consecutive weeks of meteorological and snow-cover evolution.

Each training sample was constructed using 8 consecutive weeks of observations, allowing the model to learn short-term temporal dependencies between atmospheric conditions and snow accumulation behavior.

The selected sequence length: `sequence_length = 8`

was determined experimentally after testing multiple temporal window sizes. An 8-week observation window provided the best balance between capturing recent snowfall persistence and avoiding unnecessary noise from distant past observations.

Thus, each model input sample contains:

8 weeks × 20 features

while the output corresponds to:

snow surface above 1200 m during the following week

This transforms the prediction task into a one-week-ahead temporal forecasting problem.

```
67
68 sequence_length = 8 ## sweet spot is 8 weeks after several trials
69
```

Construction of 8-week sliding temporal windows for LSTM sequence learning

8. Cyclical Representation of Week-of-Year Information

Seasonality plays a critical role in snow accumulation processes. However, the variable:

week_of_year

cannot be used directly as a linear feature because week 52 and week 1 are temporally adjacent but numerically distant.

To solve this issue, cyclical encoding was applied using sine and cosine transformations:

$$week_{sin} = \sin\left(\frac{2\pi \cdot week}{52}\right)$$

$$week_{cos} = \cos\left(\frac{2\pi \cdot week}{52}\right)$$

This transformation preserves the circular nature of seasonal time progression and allows the LSTM model to correctly interpret transitions between years.

As a result, the model can learn recurring winter snow accumulation patterns across multiple seasons.


```

11 # -----
12 # Cyclical encoding for week_of_year
13 # -----
14 df["week_sin"] = np.sin(2 * np.pi * df["week_of_year"] / 52)
15 df["week_cos"] = np.cos(2 * np.pi * df["week_of_year"] / 52)
16

```

Cyclical encoding of seasonal weekly information using sine and cosine transformations

9. Snow Memory Representation Through Lag Features

Snow accumulation is strongly dependent on previous snow presence. Unlike rainfall, snow does not disappear immediately after formation and therefore exhibits persistence over time.

To capture snow persistence behavior, three lag-based temporal memory indicators were introduced:

```

snow_last_week
snow_last_2weeks
snow_change_ratio

```

where:

```

snow_change_ratio = snow_last_week - snow_last_2weeks

```

This additional variable provides information about whether snow accumulation is increasing or decreasing between recent weeks.

Including lagged snow features allows the LSTM model to learn:

- delayed melting behavior
- accumulation continuity
- short-term snowpack memory effects

```

13 # -----
14 # Snow persistence
15 # -----
16 df["snow_last_week"] = df["s2_snow_above_1200_smoothed_zscore"].shift(1)
17 df["snow_last_2weeks"] = df["s2_snow_above_1200_smoothed_zscore"].shift(2)
18 df["snow_change_ratio"] = df["snow_last_week"] - df["snow_last_2weeks"]

```

Snow persistence feature construction using one-week, two-week and change ratio lag variables

10. Multi-Altitude Snow Aggregation Feature

Snow distribution varies across elevation ranges due to temperature gradients and precipitation differences.

To provide the model with a generalized representation of snow surface behavior across all altitude bands, a new feature was computed:

snow_mean

This variable represents the average snow surface area across:

- above 1200 m
- between 1200 m and 1800 m
- above 1800 m

It helps the model capture large-scale snowpack evolution rather than relying exclusively on individual elevation signals.

```
25 # -----
26 # Add altitude-averaged snow feature
27 # -----
28 df["snow_mean"] = (
29     df["s2_snow_above_1200_smoothed_zscore"]
30     + df["s2_snow_1200_1800_smoothed_zscore"]
31     + df["s2_snow_above_1800_smoothed_zscore"]
32 ) / 3
33
```

Altitude-averaged snow surface feature capturing global snowpack evolution

11. Snow Surface Change Rate Features

To help the LSTM model detect whether snow accumulation is increasing or decreasing over time, two temporal derivative-based indicators were introduced:

snow_gradient
snow_acceleration

The snow gradient represents the first-order variation of altitude-averaged snow signal (snow_mean) between consecutive weeks:

$$gradient = snow_t - snow_{t-1}$$

The snow acceleration represents the second-order variation:

$$acceleration = gradient_t - gradient_{t-1}$$

These features allow the model to detect:

- onset of snowfall events
- rapid melting periods
- stabilization phases in snow coverage

Including derivative-based indicators improves the model's ability to anticipate short-term snow evolution trends.

```
34 # -----  
35 df["snow_gradient"] = df["snow_mean"].diff().fillna(0)  
36  
37 df["snow_acceleration"] = df["snow_gradient"].diff().fillna(0)  
38 # -----  
39
```

First-order and second-order temporal altitude-averaged snow signals

12. Final Feature Vector Construction

The final input vector used for sequence construction included:

Feature Category	Variables
Seasonal encoding	week_sin, week_cos
Persistence	snow_last_week, snow_last_2weeks
Change dynamics	snow_change_ratio
Aggregated snow signal	snow_mean
Temporal derivatives	snow_gradient, snow_acceleration
Sentinel-2 snow coverage	3 altitude bands
MODIS snow coverage	3 altitude bands
Temperature	3 altitude bands
Precipitation	3 altitude bands

In total: 20 predictive features were used as input variables for the LSTM model.

```

37
38 feature_columns = [
39     "week_sin",
40     "week_cos",
41
42     "snow_last_week",
43     "snow_last_2weeks",
44     "snow_change_ratio",
45     "snow_mean",
46     "snow_gradient",
47     "snow_acceleration",
48
49     "s2_snow_above_1200_smoothed_zscore",
50     "s2_snow_1200_1800_smoothed_zscore",
51     "s2_snow_above_1800_smoothed_zscore",
52
53     "modis_snow_above_1200_smoothed_zscore",
54     "modis_snow_1200_1800_smoothed_zscore",
55     "modis_snow_above_1800_smoothed_zscore",
56
57     "temp_1200_smoothed_zscore",
58     "temp_1200_1800_smoothed_zscore",
59     "temp_1800_smoothed_zscore",
60
61     "precip_1200_smoothed_zscore",
62     "precip_1200_1800_smoothed_zscore",
63     "precip_1800_smoothed_zscore"
64 ]
65
66 target_column = "target_snow_next_week"
67

```

Final set of 20 predictive features used for sequence construction

13. Sliding Window Dataset Construction

A sliding-window approach was applied to transform the dataset into supervised learning sequences.

Each training sample contains: 8 consecutive weeks of input observations

mapped to: snow surface value during the following week

Mathematically:

$$X_t = [week_{t-7}, \dots, week_t]$$

$$y_t = snow_{t+1}$$

This formulation enables the LSTM network to learn temporal dependencies between past environmental conditions and future snow surface evolution.

After sequence generation, the dataset shapes became:

```
Train: (307, 8, 20)
Validation: (68, 8, 20)
Test: (67, 8, 20)
```

confirming correct preparation of the 3-dimensional tensors required for LSTM training.

```
● (esibml) reinataher@Reinas-MacBook-Air FYP_1 % python create_sequences.py
Sequences saved successfully
Train shape: (307, 8, 20)
Validation shape: (68, 8, 20)
Test shape: (67, 8, 20)
Number of features used: 20
○ (esibml) reinataher@Reinas-MacBook-Air FYP_1 %
```

Final LSTM input tensor dimensions after sequence generation

14. Deep Learning Model Selection

To predict weekly snow surface evolution above 1200 m altitude, a Long Short-Term Memory (LSTM) neural network was implemented using TensorFlow and Keras.

LSTM networks were selected because they are specifically designed for temporal sequence modeling and can capture:

- delayed meteorological effects
- seasonal accumulation cycles
- persistence of snowpack conditions
- nonlinear relationships between temperature, precipitation, and snow surface evolution

Unlike classical regression methods, LSTMs maintain a memory of previous observations through internal gating mechanisms, making them particularly suitable for weekly environmental forecasting.

15. Baseline Forecast Benchmark

Before training the neural network, a persistence-based baseline forecasting model was implemented in order to define a minimum performance threshold for the prediction task.

This baseline assumes that snow surface conditions remain unchanged between two consecutive weeks:

$$snow_{t+1} \approx snow_t$$

Within the sequence-based modeling framework, this persistence estimate corresponds to the most recent snow observation contained in each input window. Therefore, the baseline prediction was computed using the feature: `snow_last_week`

which represents snow surface conditions observed during the final time step of each sequence.

Mathematically: `baseline_pred = X_test[:, -1, 2]`

Baseline performance was evaluated using the Mean Absolute Error (MAE) metric on the independent test dataset.

This reference score provides a minimum benchmark that the LSTM model must outperform in order to demonstrate meaningful predictive capability beyond simple temporal persistence.

```
73
74 baseline_pred = X_test[:, -1, 2]
75 baseline_mae = np.mean(np.abs(baseline_pred - y_test))
76 print("Baseline persistence MAE:", baseline_mae)
77
```

```
(esibml) reinataher@Reinas-MacBook-Air FYP_1 % python train_lstm.py
Data loaded successfully
Train: (307, 8, 20)
Validation: (68, 8, 20)
Test: (67, 8, 20)
Baseline persistence MAE: 0.4887628299104479
Model: "sequential"
```

Baseline persistence prediction error used as reference benchmark for LSTM evaluation

16. Peak-Weighted Huber Loss Function

Instead of using classical Mean Squared Error (MSE), the model was trained using a peak-weighted Huber loss function designed to improve forecasting stability while preserving sensitivity to extreme snowfall events.

The Huber loss function combines the advantages of:

- Mean Squared Error (sensitive to large deviations)
- Mean Absolute Error (robust to outliers)

while preventing excessive penalization of rare extreme prediction errors.

The loss formulation follows:

$$L = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| < \delta \\ \delta(|y - \hat{y}| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

where: `delta = 1.0`

To improve prediction accuracy during high snow-accumulation periods, an adaptive peak-weighting mechanism was introduced:

```
weight = 1 + 1.5 * max(0, y_true)
```

This weighting strategy increases the contribution of large snow-surface observations during optimization while maintaining stable training convergence.

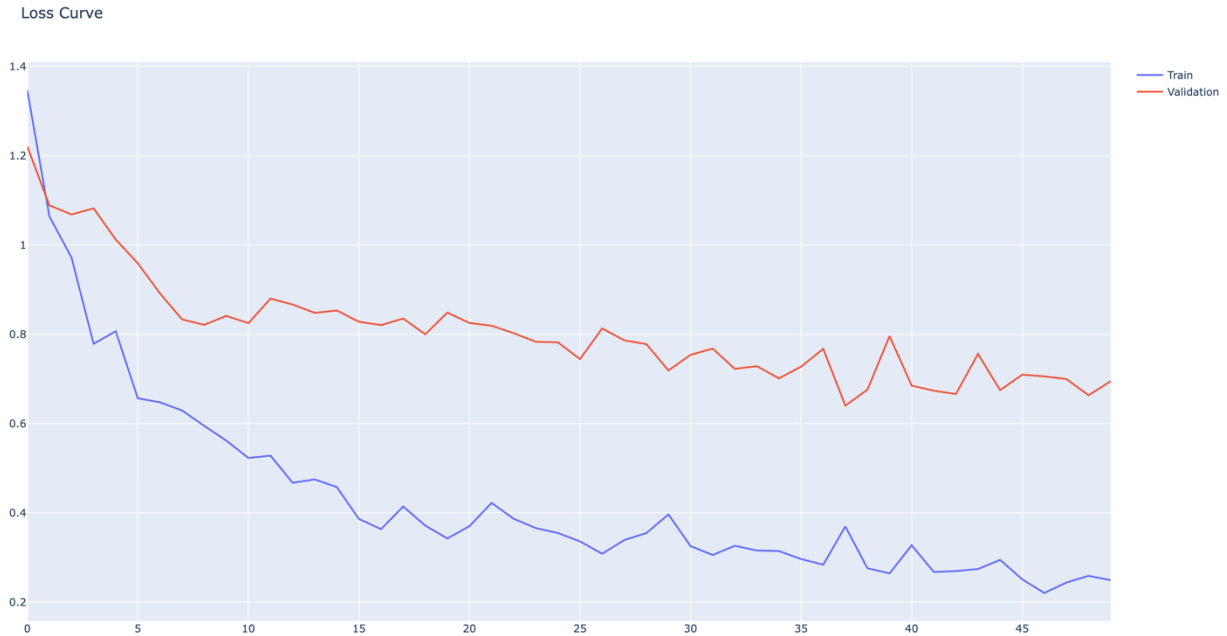
As a result, the model prioritizes learning:

- winter storm accumulation behavior
- snowpack growth episodes
- high-impact hydrological events

without introducing instability associated with aggressive threshold-based weighting strategies.

```
15
16 # =====
17 #  LOSS FUNCTION
18 # =====
19
20 def peak_weighted_huber(y_true, y_pred):
21     error = y_true - y_pred
22     abs_error = tf.abs(error)
23
24     delta = 1.0
25
26     huber = tf.where(
27         abs_error < delta,
28         0.5 * tf.square(error),
29         delta * (abs_error - 0.5 * delta)
30     )
31
32     peak_weight = 1.0 + 1.5 * tf.nn.relu(y_true)
33
34     return tf.reduce_mean(huber * peak_weight)
35
```

Implementation of the peak-weighted Huber loss function used to improve prediction stability while emphasizing high snow-accumulation events



Training and validation loss curves showing convergence behavior of the LSTM model using the peak-weighted Huber loss function

17. Multi-Layer Temporal Feature Extraction Network

To predict weekly snow surface evolution above 1200 m altitude, a stacked Long Short-Term Memory (LSTM) neural network architecture was implemented using TensorFlow and Keras.

The final architecture consists of two sequential LSTM layers followed by dense nonlinear regression layers:

Input (8 weeks × 20 features)

LSTM(96)

Dropout(0.15)

LSTM(64)

Dropout(0.15)

Dense(32)

Dense(16)

Dense(1)

Architecture rationale:

Layer	Role
LSTM(96)	capture broad seasonal dependencies
LSTM(64)	refine intermediate temporal structures
Dense layers	nonlinear regression mapping

Stacking multiple LSTM layers improves hierarchical learning of temporal patterns across different temporal scales while maintaining training stability on a moderate-sized environmental dataset.

```
91 model = Sequential([
92     Input(shape=(X_train.shape[1], X_train.shape[2])),
93
94     LSTM(96, return_sequences=True),
95     Dropout(0.15),
96
97     LSTM(64),
98     Dropout(0.15),
99
100    Dense(32, activation='relu'),
101    Dense(16, activation='relu'),
102
103    Dense(1)
104 ])
105
106 model.compile(
107     optimizer=Adam(learning_rate=0.0003),
108     loss=peak_weighted_huber, # ONLY FIXED PART
109     metrics=['mae']
110 )
111
112 model.summary()
```

Summary of the stacked LSTM architecture used for weekly snow surface forecasting

18. Overfitting Prevention Strategy

To prevent overfitting due to limited dataset size and strong temporal correlations between observations, dropout regularization was applied after each LSTM layer:

Dropout(0.15)

Dropout randomly disables 15% of neurons during each training iteration, forcing the network to learn more robust feature representations rather than memorizing training sequences.

This improves generalization performance on validation and test datasets.

19. Learning Rate Adjustment During Training

Training stability was improved using the ReduceLROnPlateau callback:

```
factor = 0.5  
patience = 6
```

If validation performance stops improving, the learning rate is automatically reduced by half.

This allows the model to:

- converge faster initially
- fine-tune parameters during later training stages
- avoid oscillation near local minima

```
120 reduce_lr = ReduceLROnPlateau(  
121     monitor='val_loss',  
122     factor=0.5,  
123     patience=6,  
124     min_lr=1e-5,  
125     verbose=1  
126 )
```

Adaptive learning-rate scheduling using ReduceLROnPlateau callback

20. Training Termination Strategy

To prevent unnecessary training beyond convergence and avoid overfitting, early stopping was applied:

```
patience = 12  
restore_best_weights = True
```

Training stops automatically when validation performance no longer improves, and the best-performing parameter configuration is restored.

This ensures optimal model generalization without manual tuning.

```
128 early_stop = EarlyStopping(  
129     monitor='val_loss',  
130     patience=12,  
131     restore_best_weights=True,  
132     verbose=1  
133 )
```

Early stopping strategy restoring best validation-performance model weights

21. Optimization Parameters

The model was trained using:

Parameter	Value
Optimizer	Adam
Learning rate	0.0003
Batch size	8
Epoch limit	150
Shuffle	Disabled

Shuffling was intentionally disabled:

```
shuffle = False
```

to preserve chronological ordering of temporal sequences during training.

This is essential for sequence-based forecasting models such as LSTMs.

22. Forecast Accuracy Measurement

Model performance was evaluated using:

Mean Absolute Error (MAE)

MAE was selected because it:

- provides interpretable prediction error magnitude
- is robust to outliers
- reflects real forecasting deviation in snow surface estimation

Evaluation was performed on the independent test dataset after training completion.

```
Restoring model weights from the end of the best epoch: 69.  
10/10 ————— 0s 9ms/step  
3/3 ————— 0s 3ms/step  
3/3 ————— 0s 4ms/step  
3/3 ————— 0s 4ms/step - loss: 0.3571 - mae: 0.4479  
  
Test MAE (scaled): 0.4478709101676941  
Test MAE (real): 0.4478708974876436
```

Test Mean Absolute Error (MAE) obtained by the LSTM model on the unseen testing dataset

23. Full Timeline Prediction Visualization

To evaluate the temporal consistency of the forecasting model across the entire observation period, a full-timeline prediction visualization was generated using Plotly.

This visualization displays:

- ground-truth snow surface values for the training dataset
- ground-truth snow surface values for the validation dataset
- ground-truth snow surface values for the test dataset
- predicted snow surface values for validation sequences
- predicted snow surface values for test sequences

The timeline representation allows visual assessment of the model's ability to reproduce seasonal snow accumulation patterns across multiple winters.

Unlike isolated statistical metrics such as MAE, this visualization provides qualitative insight into:

- seasonal prediction accuracy
- peak snowfall tracking capability
- melting-period behavior reproduction
- long-term temporal consistency

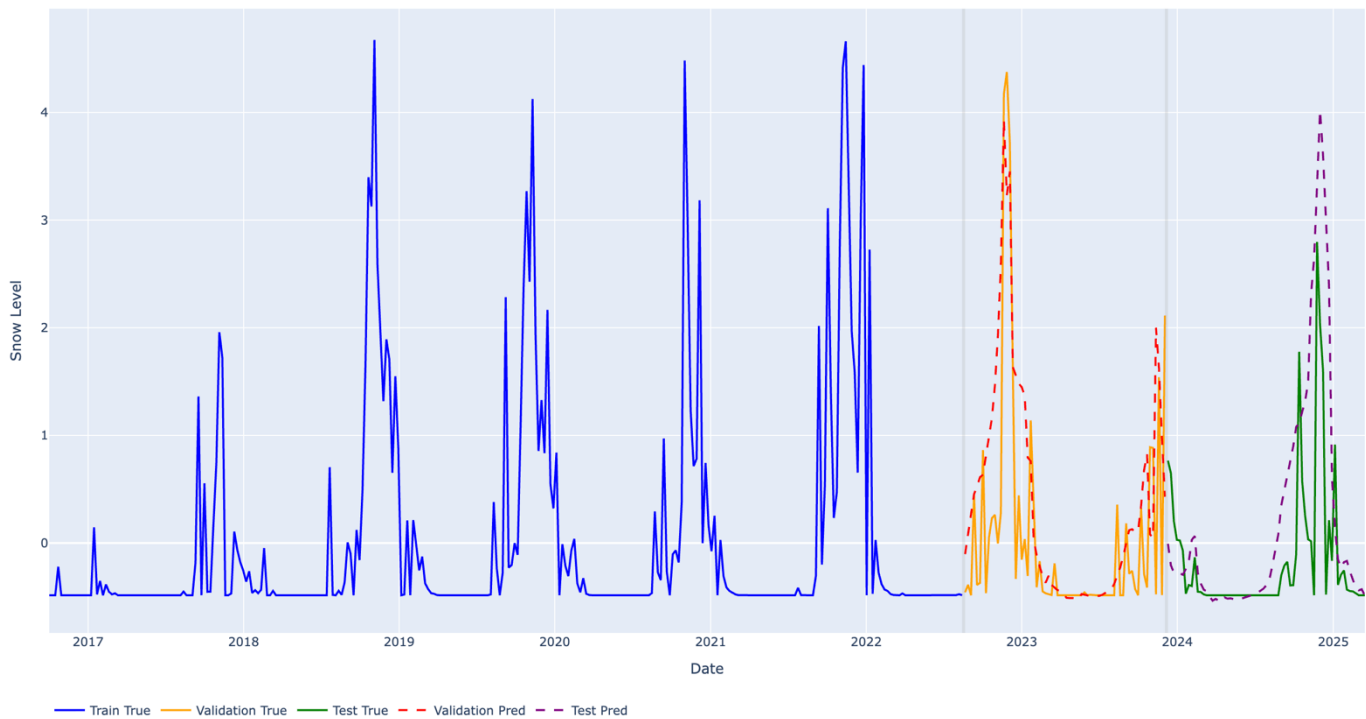
To clearly separate the different learning phases, vertical transition bands were inserted between:

- training period
- validation period
- test period

This prevents misinterpretation of prediction continuity across dataset boundaries and ensures transparency in model evaluation.

The resulting visualization confirms that the LSTM model successfully captures the main temporal structure of snow surface evolution across the observation window.

Snow Forecast (Full Timeline)



Full-timeline visualization comparing observed and predicted snow surface evolution across training, validation, and testing periods

24. Trade-Off Between Global MAE and Peak Snow Event Prediction

Although Mean Absolute Error (MAE) is commonly used to evaluate regression performance, it does not fully reflect the ability of a forecasting model to reproduce extreme snowfall events.

In the context of weekly snow surface prediction, correctly capturing seasonal accumulation peaks is particularly important because these periods correspond to the highest snow storage conditions and therefore have stronger hydrological and environmental significance.

During model tuning, it was observed that configurations producing the lowest MAE values tended to smooth the predicted signal and underestimate major snow accumulation peaks. Conversely, models trained using the peak-weighted Huber loss function slightly increased the overall MAE but significantly improved the reproduction of high-magnitude snowfall events.

This behavior reflects a known trade-off in time-series forecasting between:

- minimizing average prediction error across all weeks
- preserving accurate prediction of extreme seasonal events

Since the objective of this study is to model realistic snow surface evolution rather than only optimize a global statistical indicator, priority was given to configurations that improved peak detection performance.

As a result, a slightly higher MAE value was accepted in exchange for better reconstruction of seasonal snow accumulation dynamics, especially during winter maxima.

This choice improves the physical interpretability and operational relevance of the forecasting model.

```
Full-time predictions from
(esibml) reinataher@Reinas-MacBook-Air FYP_1 % python train_lstm.py
Data loaded successfully
Train: (307, 8, 20)
Validation: (68, 8, 20)
Test: (67, 8, 20)
Baseline persistence MAE: 0.4887628299104479
Model: "sequential"

Restoring model weights from the end of the best epoch: 69.
10/10 ██████████ 0s 9ms/step
3/3 ██████████ 0s 3ms/step
3/3 ██████████ 0s 4ms/step
3/3 ██████████ 0s 4ms/step - loss: 0.3571 - mae: 0.4479

Test MAE (scaled): 0.4478709101676941
Test MAE (real): 0.4478708974876436
```

Test Mean Absolute Error (MAE) obtained by the LSTM model (Baseline and Test)

25. Exploratory Multi-Altitude Output Prediction

In addition to the single-output forecasting model, an exploratory experiment was conducted to evaluate the feasibility of predicting snow surface evolution simultaneously at three altitude levels using a multi-output LSTM architecture.

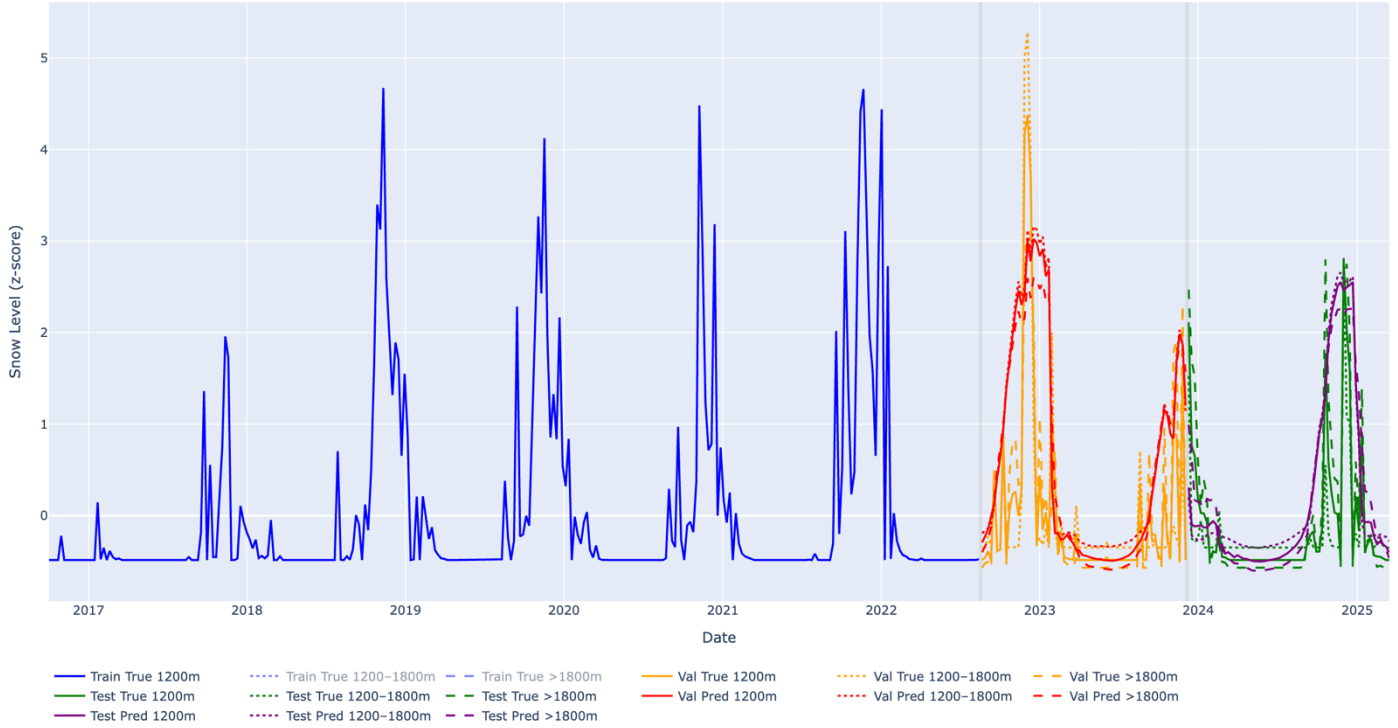
In this configuration, the network was trained to produce three predictions per time step corresponding to snow surface values at different elevations. The objective of this experiment was to investigate whether joint learning across altitudes could improve prediction consistency along the vertical snow profile.

However, the resulting predictions showed reduced interpretability compared to the single-output configuration, particularly in terms of peak reconstruction accuracy and temporal stability. While the model was able to capture general seasonal trends, the predictions were less precise for individual altitude levels.

For this reason, the single-output LSTM configuration was retained as the primary forecasting model, as it provided clearer and more reliable prediction performance.

Nevertheless, this exploratory experiment confirms the potential of multi-altitude joint prediction as a direction for future model improvement.

Snow Forecast (Multi-Altitude Model)



Example prediction results obtained using the exploratory multi-output LSTM model for simultaneous forecasting at three altitude levels

26. Conclusion

In this phase of the project, we developed a complete LSTM-based framework for weekly snow surface forecasting using multi-source satellite and meteorological data. The dataset was enhanced with temporal, persistence, and altitude-based features, and structured into 8-week input sequences for supervised learning. A stacked LSTM model was trained using a custom peak-weighted Huber loss and evaluated against a persistence baseline.

The final model achieved a test MAE of approximately 0.44, slightly outperforming the baseline MAE of 0.48, confirming the added predictive value of the proposed approach. An additional exploratory multi-output model for simultaneous altitude prediction was also tested, but the single-output configuration was retained due to its clearer and more stable performance.